

NUMERICAL PYTHON · 入门参考指南

Python 数值计算 基础语法指南

从 Python 语法、NumPy 数组到线性代数、k-path 与能带绘图

面向物理与数值计算初学者

版本 1.1 · 2026-09-02 · Python 3 · NumPy 扩充版

如何使用这份指南

这份文档不是面向 Web 开发或大型软件工程的 Python 百科，而是为数值计算建立一套够用、可靠、可继续扩展的语法基础。示例优先使用数组、矩阵、复数、本征值、动量空间采样和能带绘图。

建议 按章节顺序阅读到 NumPy 广播；之后可以按任务查阅。每段代码都建议亲自运行，并打印 shape、dtype 和少量数值进行验证。

- 1. 先确认虚拟环境和解释器正确，再运行脚本。
- 2. 先写清数组的 shape，再推导矩阵运算。
- 3. 先用小规模数据验证，再增加 k 点、矩阵维数和参数扫描范围。
- 4. 能带图使用高对称路径；严格判断带隙时扫描整个布里渊区。

内容路线

阶段	章节	你将掌握的核心对象
Python 基础	1–6	缩进、变量、容器、切片、循环、函数、模块与虚拟环境
NumPy 核心	7-10	ndarray、索引、dtype、内存、axis、广播、向量化、线性代数
数值工作流	11–15	采样、k-path、绘图、SciPy、数据读写与调试
综合应用	16–17	双层 QWZ 能带案例、速查表、练习与下一步

1. 运行 Python：解释器、脚本与虚拟环境

1.1 脚本如何被执行

Python 脚本是以 .py 结尾的文本文件。终端中最明确的运行方式是把脚本交给当前 Python 解释器：

```
python path/to/script.py
```

如果脚本具有执行权限，并且第一行 shebang 正确，也可以直接运行：

```
#!/usr/bin/env python3

./script.py
```

常见错误 shebang 必须写成 `#!/usr/bin/env python3`；如果漏掉 `usr` 前面的 `/`，系统会报告 `bad interpreter`。

1.2 激活虚拟环境

你的项目虚拟环境位于 `python/.venv`。站在 `tbq` 根目录时：

```
source python/.venv/bin/activate
which python
python --version
```

提示符通常会出现 `(.venv)`。退出时使用：

```
deactivate
```

1.3 文件命名与模块遮蔽

脚本文件名可以自由选择，但不要与标准库或第三方包同名。当前目录通常位于模块搜索路径前面，同名脚本会挡住真正的模块。

避免的文件名	冲突对象	推荐名称
array.py	标准库 array	numpy_array_basics.py
random.py	标准库 random	random_sampling_demo.py
math.py	标准库 math	math_practice.py
numpy.py	NumPy 包	numpy_basics.py
matplotlib.py	Matplotlib 包	plot_basics.py

诊断线索 报错出现 `partially initialized module` 或 `circular import` 时，先检查脚本名是否遮蔽了被导入的模块。

2. Python 最基本的语法规则

2.1 缩进决定代码块

Python 不使用花括号划分代码块，而使用缩进。冒号：表示下面将开始一个缩进块；惯例是每层使用 4 个空格。

```
x = 3

if x > 0:
    print("x is positive")
else:
    print("x is not positive")
```

规则 同一代码块必须保持相同缩进。不要混用 Tab 和空格。

2.2 注释、名称与赋值

```
# 单行注释
mass = 2.0      # 浮点数
num_k = 300     # 整数
model_name = "QWZ"
is_hermitian = True
```

- 变量名区分大小写：mass 和 Mass 是两个名称。
- 推荐 snake_case: band_energy、k_path、num_points。
- 不要覆盖常用函数名：list、sum、min、max、range。
- 赋值 = 与相等比较 == 不同。

2.3 常用运算符

类别	运算符	示例与含义
算术	+ - * /	四则运算；/ 总是产生浮点除法
整除与余数	// %	7 // 3 == 2; 7 % 3 == 1
幂	**	2 ** 3 == 8; 不要把 ^ 当成乘方
比较	< <= > >= == !=	结果为 True 或 False

类别	运算符	示例与含义
逻辑	and or not	组合布尔条件
NumPy 逐元素逻辑	& ~	数组条件必须加括号: $(a > 0) \& (a < 1)$

3. 数值类型与复数

类型	示例	数值计算中的用途
int	<code>n = 100</code>	循环次数、数组尺寸、索引
float	<code>theta = 1.05</code>	物理参数和一般实数计算
complex	<code>z = 1 + 2j</code>	量子力学、傅里叶变换、厄米矩阵
bool	<code>flag = True</code>	条件判断、掩码
None	<code>result = None</code>	表示尚无结果或可选值缺省

3.1 浮点数不是精确实数

```
x = 0.1 + 0.2
print(x)           # 0.30000000000000004
print(np.isclose(x, 0.3)) # True
```

数值比较通常使用 `np.isclose` 或 `np.allclose`，而不是要求浮点结果逐位完全相等。

3.2 复数与共轭

```
z = 1.0 + 2.0j
print(z.real)    # 1.0
print(z.imag)    # 2.0
print(z.conjugate())
print(abs(z))
```

NumPy 中可使用 `np.conj(z)`。矩阵的共轭转置通常写成 `H.conj().T`。

4. 容器、索引与切片

4.1 list、tuple、dict

```
values = [1.0, 2.0, 3.0]      # list, 可修改
point = (np.pi, 0.0)        # tuple, 常用于固定组合
params = {"m": 2.0, "tp": 0.0} # dict, 键值映射

values.append(4.0)
print(params["m"])
```

区别 Python list 是通用容器；NumPy ndarray 是为同类型数值和批量运算设计的。大型数值运算优先使用 ndarray。

4.2 索引从 0 开始

```
a = [10, 20, 30, 40]

print(a[0])    # 10, 第一个
print(a[-1])   # 40, 最后一个
print(a[1:3])  # [20, 30]
```

切片 `a[start:stop]` 左闭右开：包含 `start`，不包含 `stop`。省略 `start` 或 `stop` 表示从头开始或一直取到末尾。

写法	含义	结果 (<code>a=[10,20,30,40]</code>)
<code>a[:2]</code>	从开头取到索引 2 之前	<code>[10, 20]</code>
<code>a[2:]</code>	从索引 2 取到末尾	<code>[30, 40]</code>
<code>a[::2]</code>	步长为 2	<code>[10, 30]</code>
<code>a[::-1]</code>	反向	<code>[40, 30, 20, 10]</code>

5. 条件、循环与推导式

5.1 if / elif / else

```
if m < -2:
    phase = "trivial"
elif -2 < m < 0:
    phase = "topological"
else:
    phase = "boundary or another region"
```

5.2 for 与 range

```
for n in range(4):
    print(n)

# 输出: 0, 1, 2, 3
```

`range(4)` 也是左闭右开，相当于整数区间 $[0, 4)$ 。常见形式为 `range(start, stop, step)`。

```
for index, value in enumerate([10, 20, 30]):
    print(index, value)

for kx, ky in k_path:
    print(kx, ky)
```

第二个循环利用序列解包：`k_path` 每一行有两个数，分别赋给 `kx` 和 `ky`。

5.3 列表推导式

```
squares = [x**2 for x in range(5)]

bands = [
    np.linalg.eigvalsh(bilayer_qwz(kx, ky))
    for kx, ky in k_path
]
bands = np.array(bands)
```

推导式适合简短、明显的转换。逻辑复杂时使用普通 `for` 循环更容易调试。

6. 函数、参数与作用域

6.1 定义与返回

```
def energy(kx, ky, m=2.0):  
    dx = np.sin(kx)  
    dy = np.sin(ky)  
    dz = m + np.cos(kx) + np.cos(ky)  
    return np.sqrt(dx**2 + dy**2 + dz**2)
```

- def 后面是函数名和参数；末尾冒号后进入缩进块。
- m=2.0 是默认参数；调用时不提供 m 就使用 2.0。
- return 把结果交回调回位置，并结束当前函数。
- 没有显式 return 的函数会返回 None。

6.2 位置参数与关键字参数

```
energy(0.0, np.pi)  
energy(0.0, np.pi, 1.5)  
energy(kx=0.0, ky=np.pi, m=1.5)
```

关键字参数中的等号通常不加空格：axis=0、endpoint=False。普通变量赋值通常写成 N = 100。两种空格形式 Python 都能解析，但遵循惯例会更易读。

6.3 主程序入口

```
def main():  
    print("start calculation")  
  
if __name__ == "__main__":  
    main()
```

直接运行脚本时 __name__ 等于 "__main__"；被其他脚本 import 时不会自动执行 main()。这能把可复用函数与一次性运行流程分开。

7. NumPy ndarray：数值计算的核心

7.1 创建数组

函数	示例	用途
np.array	np.array([1, 2, 3])	从现有数据创建数组
np.zeros	np.zeros((4, 4), dtype=complex)	创建零矩阵
np.ones	np.ones(5)	创建全 1 数组
np.eye	np.eye(2, dtype=complex)	单位矩阵
np.linspace	np.linspace(-np.pi, np.pi, 101)	固定点数的均匀采样
np.arange	np.arange(0, 10, 2)	固定步长；浮点端点可能不直观

7.2 shape、ndim、size、dtype

```
a = np.array([[1, 2, 3],
              [4, 5, 6]], dtype=float)

print(a.shape) # (2, 3): 2 行 3 列
print(a.ndim)  # 2: 二维
print(a.size)  # 6: 共 6 个元素
print(a.dtype) # float64 (平台/设置可能影响具体名称)
```

第一调试原则 遇到矩阵维度问题，先打印每个中间量的 shape。

7.3 二维索引

```
print(a[0, 1]) # 第 0 行第 1 列
print(a[0, :]) # 第 0 行，所有列
print(a[:, 1]) # 所有行，第 1 列
print(a[:, 1:]) # 所有行，从第 1 列到末尾
```

逗号分隔不同轴的索引。冒号：表示该轴上的全部元素；它本身不表示左闭右开，只有带边界的 start:stop 才体现切片端点规则。

7.4 None / np.newaxis

```
v = np.array([1.0, 2.0])      # shape (2,)
row = v[None, :]             # shape (1, 2)
col = v[:, None]             # shape (2, 1)

# 完全等价:
row2 = v.reshape(1, 2)
col2 = v.reshape(2, 1)
```

None 在 NumPy 索引中表示插入一个长度为 1 的新轴；它不是区间端点。np.newaxis 与 None 完全等价。

7.5 dtype、类型转换与数值范围

一个 ndarray 通常只保存一种 dtype。dtype 决定每个元素的存储方式、内存占用以及运算精度。数值物理中建议显式区分整数、浮点数和复数。

```
a_int = np.array([1, 2, 3], dtype=np.int64)
a_float = a_int.astype(np.float64)
z = np.array([1+2j, 3-1j], dtype=np.complex128)

print(a_int.itemsize) # 每个元素的字节数
print(a_float.dtype)  # float64
print(z.real, z.imag)
```

dtype	典型用途	注意
bool	掩码和逻辑判断	每个元素只有 True / False
int64	索引、格点编号、计数	不能表示小数
float64	一般双精度数值计算	约 15-16 位十进制有效数字
complex128	量子态、厄米矩阵	实部和虚部各为 float64

常见陷阱 整数数组不会因为后来赋入小数就自动变成浮点数组。创建哈密顿量时常用 dtype=complex，避免虚部被丢弃。

7.6 赋值、视图与副本

```
a = np.arange(6).reshape(2, 3)
view = a[:, :2]          # 通常与 a 共享内存
copied = a[:, :2].copy()

view[0, 0] = 99
print(a)                 # a 也被修改
print(np.shares_memory(a, view)) # True
print(np.shares_memory(a, copied)) # False
```

`b = a` 只会让两个名字指向同一个数组。普通切片通常产生视图；花式索引和布尔索引通常产生副本。需要独立数据时显式调用 `.copy()`。

```
b = a                    # 同一个对象
r = a.ravel()            # 尽可能返回视图
f = a.flatten()          # 总是返回副本

print(a.flags.c_contiguous)
print(a.nbytes)          # 数组数据占用的字节数
```

7.7 布尔索引与花式索引

```
x = np.array([-2.0, -0.5, 0.0, 1.0, 3.0])
mask = x > 0

positive = x[mask]        # [1.0, 3.0]
selected = x[[0, 4]]      # 按索引选择
clipped = np.where(mask, x, 0.0) # 条件为假时填 0

print(np.count_nonzero(mask))
print(np.any(x < 0), np.all(x >= -2))
```

布尔数组的 `shape` 必须与被筛选的轴兼容。`x[mask]` 会把满足条件的元素取出为一维数组；`np.where(condition, a, b)` 则保留广播后的整体形状。

7.8 reshape、转轴与压缩维度

```
a = np.arange(12)
M = a.reshape(3, 4)      # shape (3, 4)
P = M.reshape(-1, 2)     # -1 让 NumPy 自动推断
MT = M.T                 # shape (4, 3)
flat = M.ravel()         # 展平

x = np.zeros((1, 3, 1))
print(np.squeeze(x).shape) # (3,)
```

函数	作用	是否改变元素总数
reshape	重新解释各轴长度	不改变
transpose / .T	交换轴顺序	不改变
squeeze	删除长度为 1 的轴	不改变
expand_dims	在指定位置插入新轴	不改变
moveaxis	把某一轴移动到新位置	不改变

7.9 concatenate、stack 与 block

```
a = np.array([[1, 2], [3, 4]])
b = np.array([[5, 6], [7, 8]])

rows = np.concatenate([a, b], axis=0) # shape (4, 2)
cols = np.concatenate([a, b], axis=1) # shape (2, 4)
batch = np.stack([a, b], axis=0)      # shape (2, 2, 2)
```

concatenate 沿已有轴连接，因此除连接轴外其他维度必须相同；stack 会先创建一个新轴。vstack 和 hstack 是常见便利函数，但复杂代码中显式写 axis 通常更清楚。

```
zero = np.zeros_like(a)
block_matrix = np.block([
    [a, zero],
    [zero, b ],
])
print(block_matrix.shape) # (4, 4)
```

7.10 推荐的随机数生成器

```
rng = np.random.default_rng(12345)

normal = rng.standard_normal((3, 4))
uniform = rng.uniform(-1.0, 1.0, size=5)
indices = rng.integers(0, 10, size=6)
choice = rng.choice([0, 1, 2], size=3, replace=False)
```

固定 seed 可以复现实验；不同 seed 产生不同序列。新代码推荐使用 Generator，而不是依赖全局的 np.random.seed 状态。

8. 广播与向量化

8.1 广播规则

NumPy 从最后一个维度向前比较形状。两个维度相同，或其中一个等于 1 时，可以广播。

```
column = np.array([[1.0],
                  [2.0],
                  [3.0]]) # shape (3, 1)
row = np.array([[10.0, 20.0]]) # shape (1, 2)

result = column + row # shape (3, 2)
# [[11, 21],
#  [12, 22],
#  [13, 23]]
```

形状 A	形状 B	能否广播	结果
(3, 1)	(1, 2)	可以	(3, 2)
(100, 2)	(2,)	可以	(100, 2)
(4, 3)	(3,)	可以	(4, 3)
(4, 3)	(4,)	不可以	尾部维度 3 与 4 冲突

8.2 segment 公式

```
def segment(start, end, n, endpoint=False):
    t = np.linspace(0.0, 1.0, n, endpoint=endpoint)
    return start[None, :] + t[:, None] * (end - start)[None, :]
```

形状依次是 $\text{start}[\text{None}, :] \rightarrow (1, 2)$, $t[:, \text{None}] \rightarrow (n, 1)$, 方向向量 $\rightarrow (1, 2)$ 。广播后得到 $(n, 2)$, 即 n 个二维 k 点。

对于当前 NumPy, 也可以直接写成更容易理解的版本:

```
def segment(start, end, n, endpoint=False):
    return np.linspace(start, end, n, endpoint=endpoint)
```

8.3 向量化优先于 Python 循环

```
# Python 循环
y_loop = []
for x in x_values:
    y_loop.append(np.sin(x)**2)

# NumPy 向量化
y = np.sin(x_values)**2
```

向量化通常更简洁、更快, 也更容易看出数学结构。但矩阵维度很大时仍需关注临时数组的内存占用。

8.4 axis 与 keepdims

```
A = np.arange(12).reshape(3, 4)

s0 = np.sum(A, axis=0)      # shape (4,)
s1 = np.sum(A, axis=1)      # shape (3,)
mean = np.mean(A, axis=0, keepdims=True)
print(mean.shape)          # (1, 4)
```

对 reduction 函数而言, axis 指被消去的轴: axis=0 把第 0 轴上的各行累加, 留下列; axis=1 把每行内部的列累加, 留下行。keepdims=True 保留长度为 1 的轴, 便于后续广播。

输入 shape	操作	输出 shape
(3, 4)	sum(axis=0)	(4,)
(3, 4)	sum(axis=1)	(3,)

输入 shape	操作	输出 shape
(3, 4)	<code>mean(axis=0, keepdims=True)</code>	(1, 4)
(3, 4, 2)	<code>max(axis=-1)</code>	(3, 4)

与 Fortran 对照 NumPy 的 `axis=0` 表示第一个轴；Fortran 的 `dim=1` 也表示第一个维度。概念相同，只是编号分别从 0 和 1 开始。

8.5 ufunc：数组上的逐元素函数

`np.sin`、`np.exp`、`np.sqrt`、`np.abs`、`np.angle` 等都是通用函数 ufunc。它们逐元素计算，同时遵守广播规则，并通常在底层编译代码中循环。

```
x = np.linspace(-2.0, 2.0, 5)
y = np.exp(-x**2)
phase = np.angle(np.exp(1j*x))
bounded = np.clip(x, -1.0, 1.0)

result = np.full_like(x, np.nan)
np.divide(1.0, x, out=result, where=(x != 0))
```

不要混淆 `np.vectorize` 主要是把 Python 函数包装成数组式调用，通常不会带来真正的速度提升。优先组合现有 ufunc 和数组运算。

8.6 QWZ 色散的二维网格向量化

```
k = np.linspace(-np.pi, np.pi, 201)
KX, KY = np.meshgrid(k, k, indexing="ij")

dx = np.sin(KX)
dy = np.sin(KY)
dz = m + np.cos(KX) + np.cos(KY)
energy = np.sqrt(dx**2 + dy**2 + dz**2)

bands_grid = np.stack((-energy, energy), axis=-1)
print(bands_grid.shape) # (201, 201, 2)
```

这里没有显式遍历 `kx`、`ky`。前两个轴保存二维布里渊区网格，最后一个轴保存两条能带。对于 2×2 QWZ 模型，解析本征值 $\pm |d|$ 特别适合向量化。

8.7 性能与内存的基本习惯

```
# 已知结果 shape 时预分配
energies = np.empty((len(k_path), 4), dtype=float)
for i, (kx, ky) in enumerate(k_path):
    energies[i] = np.linalg.eigvalsh(bilayer_qwz(kx, ky))

print(energies.nbytes / 1024**2, "MiB")
```

不要在长循环中反复调用 `np.append`，因为每次都可能重新分配并复制数组。若结果尺寸未知，先将结果存入 Python list，最后一次性调用 `np.asarray`；尺寸已知时直接用 `np.empty` 或 `np.zeros` 预分配。

```
a = np.ones(1_000_000)
a *= 2.0          # 原地运算，通常少创建一个临时数组

# 对超大网格，先估算内存
bytes_needed = 201 * 201 * 4 * np.dtype(np.complex128).itemsize
```

9. 数组运算、矩阵乘法与张量积

9.1 * 与 @ 不同

```
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])

A * B    # 逐元素乘法
A @ B    # 矩阵乘法
np.matmul(A, B) # 与 A @ B 对应
```

高频错误 物理公式中的矩阵乘法通常应使用 `@`，不是 `*`。QWZ 中 `dx*sigma1` 是标量乘矩阵，因此使用 `*` 正确。

9.2 转置、共轭转置和张量积

```
A.T      # 转置
A.conj().T # 共轭转置
np.kron(A, B) # Kronecker / 张量积
```

厄米矩阵满足 $H = H.conj().T$ 。数值上可以检查：

```
assert np.allclose(H, H.conj().T)
```

9.3 用 np.block 构造分块哈密顿量

```
coupling = tp * sigma0
H = np.block([
    [h_top,      coupling],
    [coupling.conj().T, h_bottom],
])

assert H.shape == (4, 4)
assert np.allclose(H, H.conj().T)
```

np.block 的嵌套列表直接对应数学上的分块矩阵。与手动切片赋值相比，它在块数较少时更接近公式；大型稀疏系统则应使用 scipy.sparse.bmat。

9.4 einsum：用指标表达张量缩并（选学）

```
C1 = A @ B
C2 = np.einsum("ij,jk->ik", A, B)

# 一批态 psi[k, i] 的归一化
norm2 = np.einsum("ki,ki->k", psi.conj(), psi).real
```

einsum 的字符串直接描述保留和求和的指标，适合复杂张量缩并。普通矩阵乘法仍优先使用 @；只有当指标关系更清楚或能避免中间数组时再考虑 einsum。

10. NumPy 线性代数

函数	返回内容	典型用途
np.linalg.eigvalsh(H)	厄米矩阵本征值，升序	只画能带
np.linalg.eigh(H)	本征值和本征矢	Berry 相位、态分析
np.linalg.solve(A, b)	线性方程 $Ax=b$ 的解	优先于显式求逆
np.linalg.inv(A)	逆矩阵	仅在确实需要矩阵本身时使用
np.linalg.norm(x)	向量或矩阵范数	距离、误差、归一化
np.linalg.det(A)	行列式	奇异性或解析检查
np.linalg.svd(A)	奇异值分解	低秩、条件分析

10.1 为什么使用 eigvalsh

```
E = np.linalg.eigvalsh(H)
```

eigvalsh 专门用于实对称或复厄米矩阵，只返回本征值。它利用厄米结构，通常比一般的 eigvals 更稳定，并返回从小到大排列的实数本征值。末尾 h 表示 Hermitian。

检查 如果 H 理论上应当厄米，但 eigvalsh 结果异常，先检查 H 与 H.conj().T 是否 allclose。

10.2 norm 与 axis

```
delta_k = np.diff(k_path, axis=0) # shape (N-1, 2)
dk = np.linalg.norm(delta_k, axis=1) # 每一行的二维长度
```

axis=0 在 np.diff 中表示沿行方向比较相邻 k 点；axis=1 在 norm 中表示对每行的 $(\Delta k_x, \Delta k_y)$ 计算平方和再开方。

```
# 如果 delta_k = [[dx0, dy0], [dx1, dy1]]
# 那么 dk = [sqrt(dx0**2 + dy0**2),
#            sqrt(dx1**2 + dy1**2)]
```

10.3 批量线性代数

许多 np.linalg 函数接受 shape 为 $(..., M, M)$ 的一批矩阵，并把前导维度视为批次。这样可以一次计算多个哈密顿量的本征值。

```
kx_values = np.linspace(-np.pi, np.pi, 300)
H_batch = np.stack([
    bilayer_qwz(kx, 0.0)
    for kx in kx_values
])

print(H_batch.shape)          # (300, 4, 4)
energies = np.linalg.eigvalsh(H_batch)
print(energies.shape)         # (300, 4)
```

这里构造 H 的步骤仍使用 Python 推导式，但 eigvalsh 会批量处理最后两个矩阵轴。如果哈密顿量函数也支持数组输入，可以进一步消除构造阶段的循环。

10.4 solve、inv 与条件数

```
x = np.linalg.solve(A, b)
residual = np.linalg.norm(A @ x - b)
condition = np.linalg.cond(A)

print(residual, condition)
```

求 $Ax=b$ 时应优先使用 `solve`，而不是先计算 `inv(A) @ b`。`cond(A)` 很大意味着问题对舍入误差敏感；此时即使程序没有报错，结果也可能不可靠。

10.5 检查本征矢

```
E, V = np.linalg.eigh(H)

# V 的第 n 列是第 n 个本征矢
orthogonality = V.conj().T @ V
residual = H @ V - V * E[None, :]

assert np.allclose(orthogonality, np.eye(H.shape[0]))
print(np.linalg.norm(residual))
```

`V * E[None, :]` 利用广播把第 n 列乘以对应本征值 $E[n]$ 。检查正交归一性和残差，比只观察本征值更能发现矩阵构造错误。

11. 数值采样：linspace、meshgrid 与 k-path

11.1 linspace 与 endpoint

```
np.linspace(0.0, 1.0, 5, endpoint=True)
# [0.00, 0.25, 0.50, 0.75, 1.00]

np.linspace(0.0, 1.0, 5, endpoint=False)
# [0.0, 0.2, 0.4, 0.6, 0.8]
```

`endpoint=False` 可以把一段视为 $[start, end)$ ，避免拼接多段路径时高对称点重复。最后一段通常使用 `endpoint=True`，确保最终终点被包含。

11.2 方格晶格高对称路径

```
Gamma = np.array([0.0, 0.0])
X = np.array([np.pi, 0.0])
M = np.array([np.pi, np.pi])
N = 100

k_path = np.vstack([
    segment(Gamma, X, N, endpoint=False),
    segment(X, M, N, endpoint=False),
    segment(M, Gamma, N, endpoint=True),
])
```

这样 `k_path.shape` 为 (300, 2)，索引 0、100、200、299 分别对应 Γ 、X、M、 Γ 。

11.3 累计路径长度

```
delta_k = np.diff(k_path, axis=0)
dk = np.linalg.norm(delta_k, axis=1)
k_distance = np.concatenate([0.0, np.cumsum(dk)])
```

`np.cumsum(dk)` 产生累计距离；前面补一个 0，使横坐标数量与 `k_path` 点数一致。`np.concatenate` 的第一个参数是要拼接的数组序列。

```
# dk = [0.1, 0.2, 0.3]
# cumsum(dk) = [0.1, 0.3, 0.6]
# k_distance = [0.0, 0.1, 0.3, 0.6]
```

11.4 二维网格

```
kx_values = np.linspace(-np.pi, np.pi, 201)
ky_values = np.linspace(-np.pi, np.pi, 201)
KX, KY = np.meshgrid(kx_values, ky_values, indexing="xy")

print(KX.shape, KY.shape) # (201, 201)
```

高对称 k-path 适合展示能带；判断全局最小带隙、计算 Chern 数或 DOS 时，应使用整个布里渊区的二维网格。

11.5 meshgrid 的 indexing="ij" 与 "xy"

```
kx = np.array([0.0, 1.0, 2.0])
ky = np.array([10.0, 20.0])

KX_ij, KY_ij = np.meshgrid(kx, ky, indexing="ij")
KX_xy, KY_xy = np.meshgrid(kx, ky, indexing="xy")

print(KX_ij.shape) # (3, 2): [ix, iy]
print(KX_xy.shape) # (2, 3): [iy, ix]
```

若数组约定 `field[ix, iy]`，通常选择 `indexing="ij"`；二维绘图习惯把第一轴看成纵向 `y`、第二轴看成横向 `x`，因此很多绘图示例使用 `"xy"`。关键是从一开始固定约定，并打印 `shape` 验证。

12. Matplotlib 绘图

12.1 推荐面向对象范式

```
fig, ax = plt.subplots(figsize=(6, 5))

for band_index in range(bands.shape[1]):
    ax.plot(k_distance, bands[:, band_index])

ax.set_xlabel("k path")
ax.set_ylabel("Energy")
ax.set_title("Band structure")
ax.grid(True)
fig.tight_layout()
plt.show()
```

`fig` 是整张画布，`ax` 是坐标轴。`ax.plot` 每调用一次添加一条曲线。`bands[:, band_index]` 表示所有 `k` 点上的某一条能带。

12.2 高对称点刻度

```
tick_indices = [0, N, 2*N, 3*N - 1]
tick_positions = k_distance[tick_indices]

ax.set_xticks(
    tick_positions,
    [r"$\Gamma$", r"$X$", r"$M$", r"$\Gamma$"],
)

for x in tick_positions:
    ax.axvline(x, color="0.8", linewidth=0.8)
```

12.3 show 与 savefig

```
fig.savefig("band_structure.png", dpi=300, bbox_inches="tight")
plt.show()
```

plt.show() 显示当前所有打开的图；它不表示默认值为 1。普通脚本中 show 往往会打开窗口并等待关闭。第一张 figure 内部常编号为 1，但与 show 无关。保存图片通常放在 show 之前。

方法	用途
ax.plot(x, y)	曲线
ax.scatter(x, y)	散点
ax.imshow(data)	规则网格图像/矩阵
ax.pcolormesh(X, Y, Z)	二维场
ax.contour(X, Y, Z)	等高线、费米面
ax.legend()	图例
fig.colorbar(image, ax=ax)	颜色条

13. SciPy: NumPy 之上的数值算法

NumPy 提供数组和基础线性代数；SciPy 提供积分、优化、稀疏矩阵、插值、信号处理等更高层算法。

13.1 积分

```
from scipy.integrate import quad

value, error = quad(lambda x: np.exp(-x**2), -np.inf, np.inf)
print(value, error)
```

13.2 求根与优化

```
from scipy.optimize import root_scalar, minimize

root = root_scalar(lambda x: np.cos(x) - x, bracket=[0.0, 1.0])
print(root.root)

result = minimize(lambda x: (x[0] - 2.0)**2, x0=[0.0])
print(result.x)
```

13.3 稀疏矩阵

```
from scipy.sparse import csr_matrix
from scipy.sparse.linalg import eigsh

H_sparse = csr_matrix(H_large)
E, V = eigsh(H_sparse, k=6, which="SM")
```

规模意识 小型致密厄米矩阵使用 `np.linalg.eigh`；大型稀疏矩阵只取少量本征值时，可使用 `eigsh`。

14. 数据读写与路径

14.1 使用 pathlib

```
from pathlib import Path

project_root = Path(__file__).resolve().parents[3]
result_dir = project_root / "results"
result_dir.mkdir(parents=True, exist_ok=True)
output = result_dir / "bands.npy"
```

Path 使用 / 组合路径，避免手工拼接斜杠。不要假设运行脚本时的当前目录永远不变。

14.2 保存与读取

```
np.save(output, bands)
bands_again = np.load(output)

np.savetxt(result_dir / "bands.dat", bands)
data = np.loadtxt(result_dir / "bands.dat")
```

格式	优点	注意
.npy	保留 shape 和 dtype, 快速	主要供 NumPy 使用
.npz	一个文件保存多个数组	使用名称读取
.txt/.dat	人类可读、跨工具	体积大, 精度和 shape 需明确
.csv	表格软件友好	不适合复杂高维数组

15. 调试数值程序

15.1 阅读 traceback

错误信息通常从调用链底部开始定位：先看最后一行的异常类型和说明，再向上找第一个属于你自己脚本的文件与行号。

- `SyntaxError`：代码无法解析，例如括号、冒号或缩进错误。
- `NameError`：名称尚未定义，常见于拼写错误。
- `TypeError`：对象类型或函数参数结构不对。
- `ValueError`：类型可接受，但数值或形状不符合要求。
- `IndexError`：数组索引超出范围。
- `ModuleNotFoundError`：解释器环境或模块路径不正确。

15.2 shape 驱动的调试

```
print("k_path:", k_path.shape)
print("H:", H.shape)
print("E:", E.shape)
print("bands:", bands.shape)

assert H.shape == (4, 4)
assert bands.shape[1] == 4
assert len(k_distance) == len(bands)
assert np.allclose(H, H.conj().T)
```

15.3 常见陷阱

现象	常见原因	检查方法
circular import	脚本与模块重名	检查 array.py、numpy.py、random.py 等
广播失败	shape 不兼容	打印两侧 shape，从尾部维度比较
matmul dimension mismatch	@ 两侧内维不相等	检查 (... , m, n) @ (... , n, p)
x and y must have same first dimension	绘图纵横长度不同	比较 len(x)、y.shape
图看起来有隙	k-path 没经过闭隙点	完整 BZ 网格搜索最小 gap
结果出现小虚部	浮点误差或矩阵不厄米	检查量级和 H-H †

16. 综合案例：双层 QWZ 能带

下面代码把本指南中的关键概念放在一起：复数矩阵、函数、厄米本征值、高对称路径、累计距离和面向对象绘图。

A. 导入、参数与 Pauli 矩阵

```
#!/usr/bin/env python3

import numpy as np
import matplotlib.pyplot as plt

m = 2.0
tp = 0.0
lam = 0.0

sigma0 = np.eye(2, dtype=complex)
sigma1 = np.array([[0, 1], [1, 0]], dtype=complex)
sigma2 = np.array([[0, -1j], [1j, 0]], dtype=complex)
sigma3 = np.array([[1, 0], [0, -1]], dtype=complex)
```

B. 单层与双层哈密顿量

```
def qwz(kx, ky):
    dx = np.sin(kx)
    dy = np.sin(ky)
    dz = m + np.cos(kx) + np.cos(ky)
    return dx * sigma1 + dy * sigma2 + dz * sigma3

def bilayer_qwz(kx, ky):
    h_top = qwz(kx + lam, ky)
    h_bottom = qwz(kx - lam, ky)

    H = np.zeros((4, 4), dtype=complex)
    H[:2, :2] = h_top
    H[2:, 2:] = h_bottom
    H[:2, 2:] = tp * sigma0
    H[2:, :2] = tp * sigma0
    return H
```

C. 高对称路径与能带

```
def segment(start, end, n, endpoint=False):
    return np.linspace(start, end, n, endpoint=endpoint)

def main():
    gamma = np.array([0.0, 0.0])
    x_point = np.array([np.pi, 0.0])
    m_point = np.array([np.pi, np.pi])
    n_per_segment = 100

    k_path = np.vstack([
        segment(gamma, x_point, n_per_segment, endpoint=False),
        segment(x_point, m_point, n_per_segment, endpoint=False),
        segment(m_point, gamma, n_per_segment, endpoint=True),
    ])

    bands = np.array([
        np.linalg.eigvalsh(bilayer_qwz(kx, ky))
        for kx, ky in k_path
    ])

    delta_k = np.diff(k_path, axis=0)
    dk = np.linalg.norm(delta_k, axis=1)
    k_distance = np.concatenate(([0.0], np.cumsum(dk)))

    tick_indices = [0, n_per_segment, 2*n_per_segment,
                    3*n_per_segment - 1]
    tick_positions = k_distance[tick_indices]
```

D. 绘图

```
fig, ax = plt.subplots(figsize=(6, 5))
for band_index in range(bands.shape[1]):
    ax.plot(k_distance, bands[:, band_index])

ax.set_xticks(
    tick_positions,
    [r"$\Gamma$", r"$X$", r"$M$", r"$\Gamma$"],
)
ax.set_ylabel("Energy")
ax.set_title("Bilayer QWZ dispersion")
ax.grid(True)
fig.tight_layout()
plt.show()
```

E. 程序入口

```
if __name__ == "__main__":
    main()
```

16.1 当前参数下为什么 gapless

当 $m=2$ 、 $tp=0$ 、 $\text{lam}=0$ 时，两层是两份相同且互不耦合的 QWZ 模型。单层闭隙点在 $M=(\pi, \pi)$ ，四个本征值在该点数值上接近 0。 Γ -X-M- Γ 路径会经过 M，因此能带图能够显示闭隙。

物理与数值的边界 标准高对称路径适合展示；若 $\text{lam} \neq 0$ 、应变或其他项降低对称性，闭隙点可能离开标准路径，应在完整二维 BZ 上搜索最小带隙。

16.2 全布里渊区最小直接带隙

```
def minimum_direct_gap(nk=201):
    k_values = np.linspace(-np.pi, np.pi, nk)
    minimum = np.inf
    minimum_k = None

    for kx in k_values:
        for ky in k_values:
            energy = np.linalg.eigvalsh(bilayer_qwz(kx, ky))
            gap = energy[2] - energy[1]
            if gap < minimum:
                minimum = gap
                minimum_k = (kx, ky)

    return minimum, minimum_k
```

这里假定半填充时第 2、3 条排序后的能带构成目标带隙。更一般的问题需要明确占据带数、费米能和直接/间接带隙的定义。

17. 速查表与练习

17.1 最常用语法速查

任务	写法
导入 NumPy	<code>import numpy as np</code>

任务	写法
创建数组	<code>a = np.array([1.0, 2.0])</code>
查看形状	<code>a.shape</code>
转换 dtype	<code>a.astype(np.float64)</code>
独立副本	<code>b = a.copy()</code>
布尔筛选	<code>a[a > 0]</code>
条件选择	<code>np.where(mask, a, b)</code>
改变形状	<code>a.reshape(m, n)</code>
所有行第 n 列	<code>a[:, n]</code>
插入行轴	<code>a[None, :]</code>
插入列轴	<code>a[:, None]</code>
沿已有轴拼接	<code>np.concatenate([a, b], axis=0)</code>
创建新批次轴	<code>np.stack([a, b], axis=0)</code>
沿轴求和	<code>np.sum(a, axis=0)</code>
保留被消去的轴	<code>np.mean(a, axis=0, keepdims=True)</code>
矩阵乘法	<code>A @ B</code>
共轭转置	<code>A.conj().T</code>
厄米本征值	<code>np.linalg.eigvalsh(H)</code>
批量厄米本征值	<code>np.linalg.eigvalsh(H_batch)</code>
均匀采样	<code>np.linspace(start, stop, num)</code>
二维网格	<code>np.meshgrid(kx, ky, indexing="ij")</code>
相邻差分	<code>np.diff(a, axis=0)</code>
累计和	<code>np.cumsum(a)</code>

任务	写法
垂直拼接	<code>np.vstack([a, b])</code>
横向拼接一维数组	<code>np.concatenate((a, b))</code>
浮点比较	<code>np.allclose(a, b)</code>
随机数生成器	<code>rng = np.random.default_rng(seed)</code>
创建图	<code>fig, ax = plt.subplots()</code>
显示图	<code>plt.show()</code>

17.2 建议练习

1. 创建 shape 分别为 (2,)、(1,2)、(2,1) 的数组，打印三者并解释区别。
2. 用 `np.linspace` 在 Γ 和 X 之间生成 5 个点，分别比较 `endpoint=True` 与 `False`。
3. 构造三个 Pauli 矩阵，验证 `sigma_i @ sigma_i` 等于单位矩阵。
4. 随机构造复矩阵 A ，并令 $H=A+A.conj().T$ ，验证 H 是厄米矩阵。
5. 使用 `eigvalsh` 和 `eigh` 计算同一 H ，比较本征值并检查本征矢归一化。
6. 比较普通切片、花式索引和 `.copy()` 是否与原数组共享内存。
7. 对 `shape=(3,4,2)` 的数组分别沿 `axis=0、1、2` 求和，并预测输出 `shape`。
8. 用布尔索引筛选随机数组中的正数，再用 `np.where` 保留原 `shape`。
9. 把 300 个 4×4 厄米矩阵堆叠为 H_{batch} ，并使用批量 `eigvalsh`。
10. 绘制 QWZ 模型 $m=-2、0、2$ 附近的能带，确认闭隙点分别位于哪些高对称点。
11. 比较高对称路径得到的最小 `gap` 与完整二维网格搜索的结果。

17.3 下一步学习顺序

- 先熟练 `ndarray` 的 `shape`、索引、`axis` 和广播。
- 再掌握线性代数、复数、厄米矩阵和本征问题。

- 随后学习 SciPy 稀疏矩阵、积分、优化和插值。
- 最后进入能带跟踪、Berry 曲率、Chern 数、DOS 和性能优化。

核心习惯 每写一个数值函数，都明确输入 shape、输出 shape、dtype、单位、参数范围和验证方法。

附录：推荐项目结构

```
python/  
├── .venv/          # 虚拟环境，不提交 Git  
├── requirements.txt # 依赖版本  
├── src/tbg/        # 可复用模块  
│   ├── geometry.py  
│   ├── hamiltonian.py  
│   └── kpath.py  
├── scripts/        # 具体计算入口  
│   └── band_structure.py  
├── tests/          # 自动测试  
└── test_hamiltonian.py
```

src 保存可复用算法，scripts 设置参数并运行具体任务，tests 检查数学性质和回归结果。随着项目扩大，可以加入 pyproject.toml，并用可编辑安装让 import 路径更加稳定。

— 完 —